Best Digits

AlgoExpert · Medium · Stacks

The problem

- Remove exactly numDigits digits from a number string to make the largest possible number.
- Monotonic decreasing stack.

Intuition

- Greedy with a stack: drop a previous digit whenever a larger one arrives, while deletions remain.

Brute force vs optimal

- Try all C(n, k) removals → exponential
- Monotonic stack: pop smaller digits while removals remain → O(n)
- Remove from the end if removals remain

Approach

- 1. Push digits
- 2. Pop smaller tops while deletions remain
- 3. Trim extra from the end

Worked example

stack (top ↑)




```
read 4

removals left: 2
```

Read 4

Worked example

stack (top ↑)




```
read 6: pop 4

removals left: 1
```

Read 6, remove 4

Worked example

stack (top ↑)




```
read 2

removals left: 1
```

Read 2

Worked example

stack (top ↑)



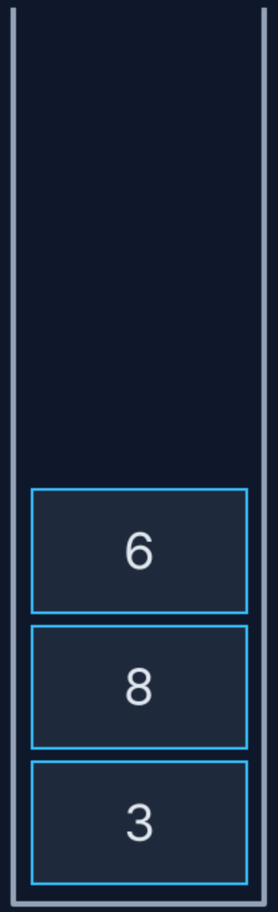

```
read 8: pop 2

removals left: 0
```

Read 8, remove 2

Worked example

stack (top ↑)




```
read 3

removals left: 0
```

Read 3

Worked example

stack (top ↑)




```
read 9

removals left: 0
```

Read 9

Java solution

```java
public class BestDigits {
    public static String bestDigits(String number, int numDigits) {
        ArrayDeque<Character> st = new ArrayDeque<>();
        int k = numDigits;
        for (char c : number.toCharArray()) {
            while (k > 0 && !st.isEmpty() && st.peekLast() < c) { st.pollLast(); k--; }
            st.addLast(c);
        }
        while (k-- > 0) st.pollLast();
        StringBuilder sb = new StringBuilder();
        for (char c : st) sb.append(c);
        return sb.toString();
    }
}
```

Complexity

- Time: O(n)
- Space: O(n)

Edge cases & pitfalls

- Leftover removals: drop from the end
- Equal digits: don't pop
- Result length is n − k

Interview tips & follow-ups

- Mirror: Remove K Digits for the smallest number

Recap

- Greedy with a stack: drop a previous digit whenever a larger one arrives, while deletions remain.
- Complexity: O(n)